Artifact of the paper "Implementing Set-Theoretic Types"

This repository contains the companion artifact for the paper "Implementing Set-Theoretic Types".

Kicking the tires

Via docker

Expected time: a few minutes (the time to download the Docker image).

The simplest way to check is to pull the image

```
$ docker run -ti --rm --name sstt-run nguyenkim/sstt:v1.0.0 bash
```

Once inside the container :

```
$ cd sstt
$ dune build
```

This will just build the library in default mode.

Via Makefile

Expected time: 20 minutes (the time to download dependencies and build a local opam switch, may vary depending on the user's hardware).

Running outside docker has only been tested on Debian/Ubuntu but should run fine on any Unix system that supports OCaml. First, one should install a few external dependencies. The list of (Ubuntu/Debian) packages is:

```
  binaryen bzip2 ca-certificates \
  cmake curl dc g++ git libcurl4-gnutls-dev \
  libexpat1-dev libgmp-dev libssl-dev \
  make ninja-build npm pkg-config python3 rsync texlive-science \
  texlive-pictures texlive-latex-extra unzip
```

the `opam` package manager should also be installed. Once this is installed, from the provided tarball of the artifact, one can run:

```
$ make setup
```

This will create a local OPAM switch with the OCaml compiler and required dependencies (so has to not interfere with an existing OCaml installation). This setup can take between 5 and 15 minutes depending on the user configuration.

The installation can then be tested by doing:

```
$ cd sstt
$ dune build
```

Which will build the library.

Paper overview

The paper reports on the SSTT (Simple Set-Theoretic Types) library, in particular the data-structures used to represent set-theoretic types. Such types are (informally) given by the grammar:

$$t ::= b | t \times t | t \to t | t \wedge t | t \vee t | \neg t | \mathbb{0} | \mathbb{1} | \alpha$$

where

- b stands for basic types
- $\vee$, $\wedge$, $\neg$ stand for union, intersection and negation of types
- $\mathbb{0}$ for the empty type
- $\mathbb{1}$ for the top type
- α is a type variable

The paper proposes a data-structure, dubbed `BDT` (for Binary Decision Tree) to represent such types. It also presents :

- a semantic simplification (SS) aimed at reducing the size of BDTs
- the complete pseudo-code of an optimized subtyping algorithm (deciding, given two polymorphic type t and s whether $\forall \sigma, t\sigma \leq s\sigma$) (which uses a cache to avoid recomputations, and whose handling is complex in the presence of recursive types)
- the complete pseudo-code of an optimized tallying algorithm (unification based subtyping constraints), which, given two types t and s, finds the principal set of substitutions such $\{\sigma_1, \dots, \sigma_n\}$ such that $t\sigma_i \leq s\sigma_i$

The present artifact allows one to reproduce the experimental results in the paper. The experiment runs in two phases:

Phase 1

The tests uses the MLsem type checker to typecheck three corpuses of programs. During typechecking, the tallying (unification) problems are extracted and serialized to json files. As the goal of the paper is *not* to benchmark a particular type checker implementation but only the type data structure and basic function of subtyping and tallying, no benchmarking is done during this first phase.

Phase 2

The tallying problems from Phase 1 are read and solved by a benchmarking program which uses the SSTT library. The following 8 configurations are tested :

- BDT : a plain BDT data-structure is used, without any optimization for size
- SS : a BDT + semantic simplification data-structure is used
- BDT + HC : a BDT + hashconsing of nodes
- SS + HC : a BDT + semantic simplification + hashconsing of nodes
- BDT + Naive Sub : a plain BDT implementation and a naive version of the subtyping algorithm (without caching)
- SS + Naive Sub : an SS implementation and a naive version of the subtyping algorithm (without caching)
- SS + Naive Tallying : an SS implementation and a naive version of the tallying, without constraint propagation nor simplification
- CDuce : an implementation where the types/subtyping/talling procedure are replaced by those of the CDuce compiler, the only known full implementation of set-theoretic types.

The paper also makes the claim that a preliminary (and more naive) version of the semantic simplification is present in the artifact of Polymorphic Type Inference for Dynamic Languages, POPL24 and that such simplifications (like the SS strategy) are necessary in practice or the type quickly grow too large to be useful.

Evaluation instructions and Reproducibility guidelines

We assume that the test are being done from inside the docker container (in the home directory of the `sstt` user) or at the root of the provided tarball, with the preliminary setup done.

Reproducing the benchmarks from p. 22

Expected time: 15 minutes (depending on the test machine).

Regarding performances of the data structure and algorithms, the paper makes the following claims, summarized in the table of p. 22:

- of the four configuration for the data-structure (BDT, SS, HC, SS+HC), SS is second only to the "ideal" BDT mode (which is not a realistic mode in practical settings). This is shown by comparing the runtime lines of the first four columns of the table and seeing that the second best total time is SS (the first one being BDT)
- the improved subtyping algorithm is an improvement for the SS mode. This is shown by the "Naive subtyping/SS" column being slower than the SS* column.
- the improved tallying algorithm is an improvement for the SS mode. This is shown by the "Naive tallying/SS" column being slower than the SS* column.
- the default SS* strategy is better than an implementation based on the CDuce compiler (comparing the CDuce column with the SS*).

The table from p. 22 of the paper can be reproduced with:

```
$ make phase2
```

This will:

- generate the JSON files containing the tallying problems for the three corpuses (the phase 1 described above)
- evaluate each corpus for all 8 configurations (the actual phase 2)

This generates in the `sstt/output` directory:

- 8 `.log` files (`01....log` to `08....log`) containing the raw numbers
- a `benchmark.tex` file which is also displayed on the terminal at the end of the test and which contains the LaTeX code for the table given page 22 of the paper. The generated table should be interpreted as such:
- Lines `Building/Solving/Total` contain absolute timing and might differ wildly from those in the paper, depending on the actual machine running the tests
- Lines `Slowdown` display the relative slowdown of all strategy w.r.t to the ideal one and should show a similar trend
- `#Sol` should give the same results as in the paper
- `Size/Avg. Size/Peak Size` show memory consumption and should give numbers very close to those in the paper
- `Timeout` shows the number of tests that were interrupted after 10s

This last line depends on the actual test machine (a faster machine than used for the paper may pass some test under 10s instead of timing out). If the `Timeout` result is different from the paper in a given configuration, then it is expected that the other numbers (including `#Sol`) also change, since more tests were executed.

The table can be visualized as PDF by going to the `sstt/output` directory and doing:

```
$ cd sstt/output
$ make table.pdf
```

a `benchmark.tex` file must be present (that is, `make phase2` needs to have been completed successfully once). This also requires a Texlive installation with the `nicematrix` package (on Debian, installing `texlive-science` and `texlive-pictures` is enough).

Note: when running from inside a container, it is recommended to extract the benchmark files outside:

```
$ docker cp sstt-run:/home/sstt/sstt/output output_sstt
```

Claim about type simplification in [POPL24]

Expected time: less than a minute

The experimental section also claims (top of page 23) that an implementation based on CDuce alone cannot perform well and that an auxiliary simplification procedure is necessary. This claim can be tested with:

```
$ make claim_popl24
```

This experiments first typechecks (using the artifact of [POPL24]) a simple file containing the `concat` and `flatten` functions on lists. Typechecking should succeeds in less than half a second. Then, the script patches the code to disable type simplification (the `simplify_typ` function is replaced by the identity). Typechecking is run again and should fail with a timeout after 10s.

Reusability guidelines

The artifact is hosted on Github and archived on Zenodo, and released under the MIT License.

The core of the artifact is the instrumented version of the SSTT library, in the `sstt` folder. It can be run in various modes by modifying the file `sstt/src/lib/sstt/core/utils/config.ml`:

```
type subtyping_cache = HashCache | MapCache | BasicCache
let use_cduce_backend = false     (* Default: false *)
let hash_consing = false          (* Default: false *)
let bdd_simpl = true              (* Default: true *)
let benchmark_size = false        (* Default: false *)
let tallying_opti = true          (* Default: true *)
let subtyping_cache = HashCache   (* Default: HashCache *)
```

And recompiling/running the benchmark program:

```
$ dune exec -- src/bin/benchmark.exe file.json
```

A JSON file has the following format:

```
[
  {
    "vars": [ "'a", "'b" ],
    "mono": [],
    "rvars": [],
    "rmono": [],
    "constr": [
      [
        "'a & lst(any, x1) where x1 = lst(tuple0 | (any, x1))",
        "lst(any, 'b & lst(x1)) where x1 = tuple0 | (any, lst(x1))"
      ]
    ]
  },
```

```
...
]
```

That is a collection of tallying problems where `vars`, `mono`, `rvars`, `rmono` are respectively polymorphic variables (can be instantiated), frozen variables (cannot be instantiated), row variables and frozen row variables. The `constr` fields contains a list of lists of pairs of types consisting of tallying problem `(t1, t2)` meaning "find all substitutions such that `t1 <= t2`". The syntax of types is documented in `sstt/REPL.md`.

Note however that:

- SSTT is under active development, and may diverge from the archived artifact. The latter is provided as an actual full-fledged implementation of the algorithms and data-structures documented in the paper.
- Even within the limits of this artifact, this is an instrumented version of the library. The instrumentation causes the code to be slightly slower than a non-instrumented version.

Furthermore, the artifact also serves as a general sandbox to play with set-theoretic types. A web based REPL (documented in `sstt/REPL`) is can be build by doing:

```
$ cd sstt
$ make web-deps js wasm
$ cd web
$ python3 -m http.server
```

or from the docker container:

```
$ docker run --name sstt-run -p 8000:8000 sstt
```

And then pointing one's web browser on https://localhost:8000. Here some problems can be tested, such as `[('A , int) <= (bool, 'B)];;` which executes the tallying algorithm to find substitutions that make the inequation hold.